

浙江大学实验报告

专业：微电子科学与工程
姓名：赵楼晗
学号：3240104430
日期：2026 年 4 月 8 日
地点：紫金港东 4-223

课程名称：数字系统设计实验
实验名称：常用组合电路模块的设计和应用

指导老师：屈民军、唐奕
实验类型：FPGA 应用实验

成绩：
同组学生姓名：张浩杰

一、实验目的和要求

1.1 实验目的

- (1) 掌握用 Verilog HDL 描述数据选择器、加法器和比较器等电路模块。
- (2) 了解“自顶而下”的数字设计方法，掌握系统层次结构的设计。
- (3) 掌握模块调用的方法，掌握参数定义和参数传递的方法。
- (4) 掌握 ModelSim 功能仿真的工作流程，进一步了解 Vivado 的工作流程。
- (5) 认识到文件管理的重要性。

1.2 实验要求

任务一：设计两数之差的绝对值电路：电路输入 a_{in} 、 b_{in} 为 4 位无符号二进制数，电路输出 out 为两数之差的绝对值，即 $out = |a_{in} - b_{in}|$ 。要求用多层次结构设计电路，即调用数据选择器、加法器和比较器等基本模块来设计电路。

任务二：设计模式比较器 (Mode-Dependent Comparator) 电路：电路的输入为两个 8 位无符号二进制数 a 、 b 和一个模式控制信号 m ；电路的输出为 8 位无符号二进制数 y 。当 $m=0$ 时， $y=MAX(a,b)$ ；而当 $m=1$ 时，则 $y=MIN(a,b)$ 。要求用多层次结构设计电路，即调用数据选择器和比较器等基本模块来设计电路。

二、实验内容和原理

2.1 设计思路分析

任务一要求设计计算两数之差的绝对值的电路，任务二要求设计模式比较器。分析发现，两数的差的绝对值可以通过比较器比较大小后交换相减位置实现，模式比较器的模式切换可以通过数据选择器实现；减法功能可以由加法补码实现，而加法可由全加器实现。由此可以设计出整体电路的框图：（图1、图2）

发现电路中使用了三个常用底层模块：**比较器**、**二选一选择器**、**加法器**。按照“自顶而下”、层次设计的核心思想，我们首先进行底层模块设计，再在顶层进行调用。

2.2 底层模块设计

2.2.1 数据比较器 Verilog HDL 描述

数据比较器的核心功能是：输入两个 n 位二进制数 a 和 b ，对两位数进行逐位比较，若 $a > b$ ，则 agb 输出高电平；若 $a = b$ ，则 aeb 输出高电平；若 $a < b$ ，则 alb 输出高电平。

对于两个数的比较与 agb 、 aeb 、 alb 的幅值在 Verilog 中可以通过 `always` 过程块结合 `if` 条件判断实现：

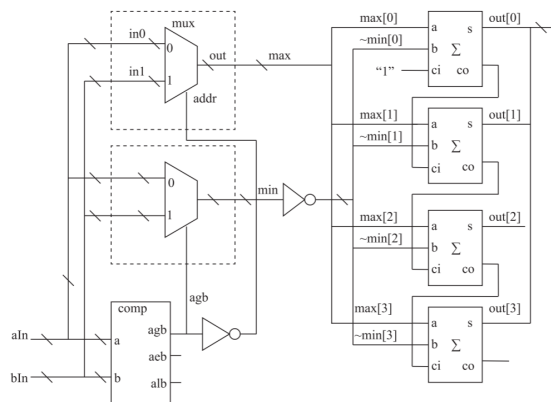


图 1: 任务一电路框图

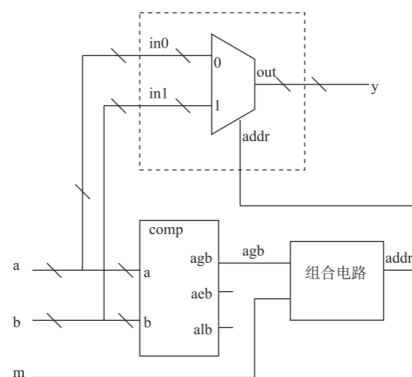


图 2: 任务二电路框图

```

1  always @(*) begin
2      if (a > b) begin
3          agb = 1;
4          aeb = 0;
5          alb = 0;
6      end
7      else if (a == b) begin
8          agb = 0;
9          aeb = 1;
10         alb = 0;
11     end
12     else begin // a < b
13         agb = 0;
14         aeb = 0;
15         alb = 1;
16     end
17 end

```

2.2.2 数据选择器 Verilog HDL 描述

两位数据选择器（mux2to1）的核心功能是：输入两个 n 位二进制数 a 和 b ，根据输入的地址位 $addr$ 选择输出； $addr=0$ 时输出 $out=a$ ， $addr=1$ 时 $out=b$ 。

在 Verilog 中可以用选择赋值语句一句话实现两位数据选择器的功能：

```

1  assign out = addr ? in1 : in0;

```

2.2.3 全加器 Verilog HDL 描述

全加器（full adder）可以实现加法的功能：输入两个一位二进制数 a 、 b ，以及上一位的进位 ci ，输出三个数相加的结果，其中 s 用于存储相加结果的个位， co 用于存储相加结果的十位（也就是下一位的进位）。

分析真值表可以发现，两个数相加，结果的个位为 $s = a \oplus b$ ，结果的十位为 $co = ab$ ，由此可以构建半加器 (half adder)；两个半加器进行级联即可构成全加器。

实现全加器的核心代码可用 assign 赋值语句配合逻辑表达式实现：

```
1 assign s = a ^ b ^ ci;
2 assign co = (a & b) | (a & ci) | (b & ci);
```

2.3 顶层 Verilog 描述

2.3.1 任务一顶层设计

任务一的顶层设计已有示例代码给出，我们只需理解并加以运用即可。

可以发现，代码主要分成三个部分：wire 型数据的定义、实例化调用底层模块、参数传递。底层模块实例化在电路中相当于我们先搭建出若干底层模块，而参数传递则相当于接线。

2.3.2 任务二顶层设计

任务二的顶层需要我们自行设计。我们同样按照任务一“数据定义、实例化调用底层模块、参数传递”的流程进行。

分析电路核心功能，我们需要比较两个数的大小，并根据 m 的值选择 MAX 或 MIN 进行输出。一种实现的思路是：首先比较两个数的大小，将比较结果 agb、alb 作为两个分别负责选择 MAX 和 MIN 的选择器的 addr，再将 m 作为选择 MAX 和 MIN 的 addr，从而得到输出。这种方法总共使用了 1 个比较器和 3 个选择器。

实现该功能的核心 Verilog 代码如下：

```
1 comp # (8) comp1(.a (a),.b (b),.agb(agb),.aeb(aeb),.alb(alb));
2 mux_2to1 # (8) mux_min (.in0 (a),.in1 (b),.addr(agb),.out (min));
3 mux_2to1 # (8) mux_max (.in0 (a),.in1 (b),.addr(alb),.out (max));
4 mux_2to1 # (8) mux_out (.in0 (max),.in1 (min),.addr(m),.out (y));
```

三、 主要仪器设备

- (1) 装有 Vivado、ModelSim SE 软件的计算机。
- (2) Nexys Video FPGA 开发板。

四、 操作方法和实验步骤

4.1 设计思路分析和 Verilog 代码编写

具体代码编写已在“实验内容和原理”部分中完整阐述。

4.2 使用 ModelSim 进行仿真

将编写好的 Verilog 代码导入对应的 ModelSim 工程中，右键老师给出的 tb 测试文件点击“simulate”开始仿真；在对象窗口中，选择该模块的所有变量加入观察窗口，在点击“simulate”选项下的“run -all”，即可观察到波形（如图3）。

确认波形无误、电路工作正常后，就可以建立 vivado 工程并烧录代码。具体各模块仿真结果见后“实验数据记录和处理”模块。

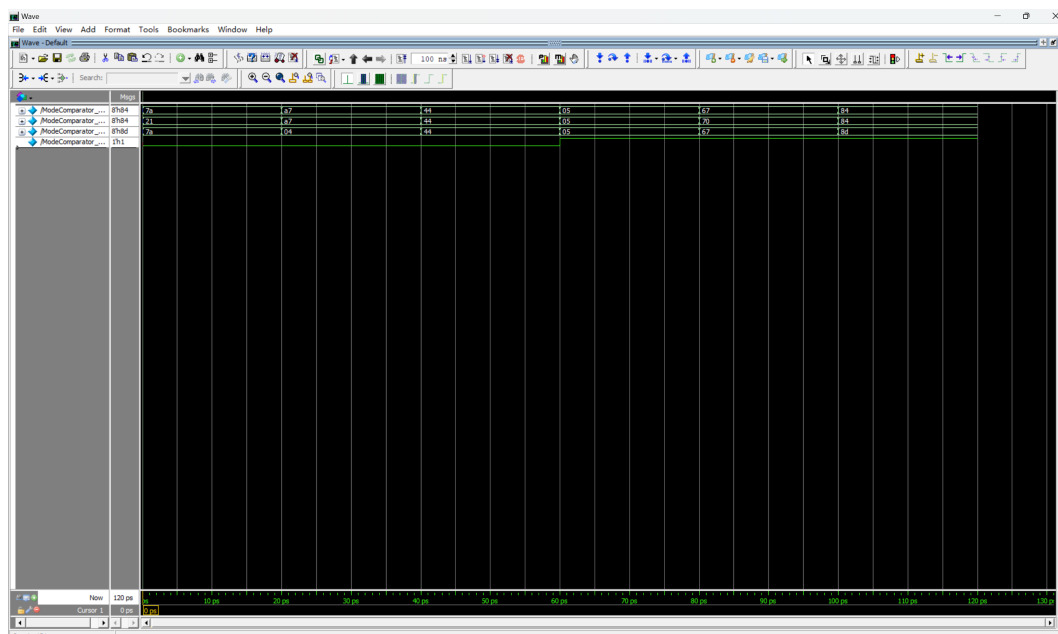


图 3: 仿真波形示例

4.2.1 创建 vivado 工程并烧录代码

打开 vivado，新建工程文件；导入源代码，进行 RTL 分析得到原理图（图4）。

再进行综合（Synthesis），得到网表；进行如下引脚约束（设置八个开关为输入，4 个 LED 灯为输出，如图5），进行实现（Implementation），将具体电路结构落实分配到 FPGA 上的门阵列中；完成后即可生成 Bitrate 文件用于直接烧录到 FPGA 上。

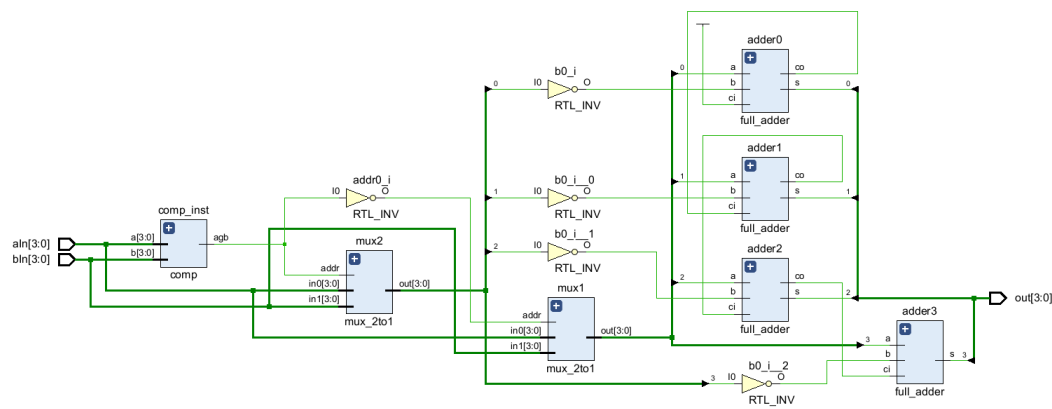


图 4: 电路原理图

最后打开烧录硬件管理器（Hardware Manager），用 USB 串口连接开发板后选择自动连接开发板（Auto Connect），并将 Bitrate 文件烧录（Program Device），等待片刻上传成功。

Tcl Console														Messages	Log	Reports	Design Runs	Package Pins	I/O Ports	x					
Name		Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM											
All ports (12)																									
ain (4) IN																									
ain[3] IN														G22		16	LVCMOS33*	3.300		NONE		NONE			
ain[2] IN														G21		16	LVCMOS33*	3.300		NONE		NONE			
ain[1] IN														F21		16	LVCMOS33*	3.300		NONE		NONE			
ain[0] IN														E22		16	LVCMOS33*	3.300		NONE		NONE			
bin (4) IN																									
bin[3] IN														M17		15	LVCMOS33*	3.300		NONE		NONE			
bin[2] IN														K13		15	LVCMOS33*	3.300		NONE		NONE			
bin[1] IN														J16		15	LVCMOS33*	3.300		NONE		NONE			
bin[0] IN														H17		15	LVCMOS33*	3.300		NONE		NONE			
out (4) OUT																									
out[3] OUT														U16		13	LVCMOS33*	3.300	12			NONE		FP_VTT_50	
out[2] OUT														T16		13	LVCMOS33*	3.300	12			NONE		FP_VTT_50	
out[1] OUT														T15		13	LVCMOS33*	3.300	12			NONE		FP_VTT_50	
out[0] OUT														T14		13	LVCMOS33*	3.300	12			NONE		FP_VTT_50	
Scalar ports (0)																									

图 5: 引脚约束

4.3 测试硬件功能

将项目代码烧录后，便可对 FPGA 进行功能测试。图6中测试计算两数之差的绝对值电路，发现电路功能基本正确（具体可见测试视频），至此可得出结论：本次设计基本符合要求，仿真结果基本正确，最终电路功能符合预期。

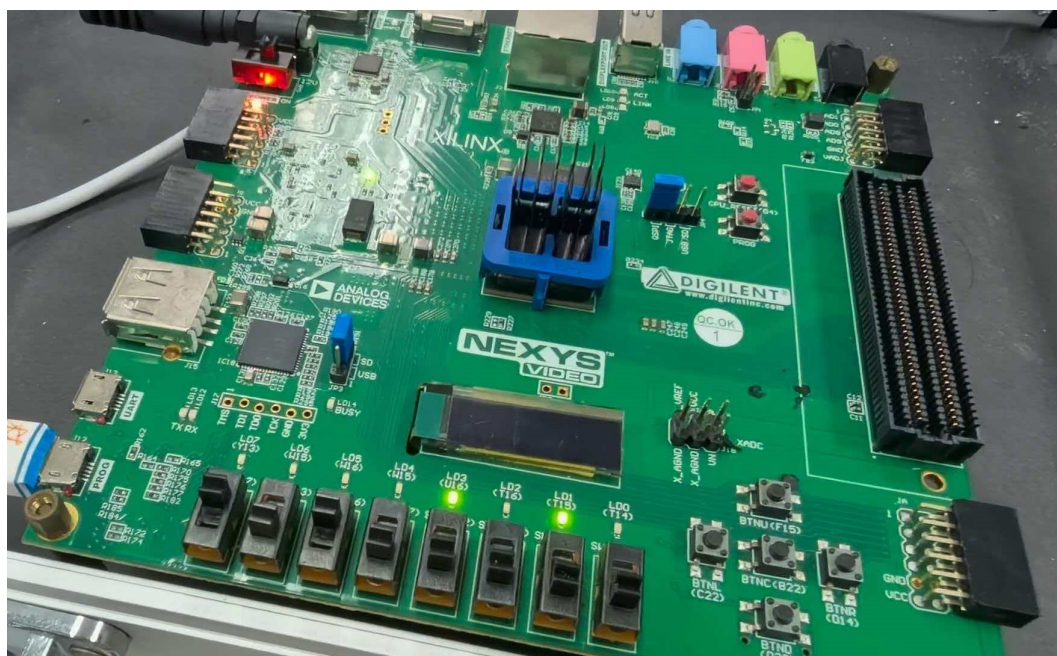


图 6: FPGA 运行测试: $|1010-0000|=1010$

五、实验数据记录和处理

本部分主要对各模块的仿真结果进行分析。

5.1 比较器模块

比较器的功能为：输入两个 n 位二进制数 a 和 b ，对两位数进行逐位比较，若 $a > b$ ，则 agb 输出高电平；若 $a = b$ ，则 aeb 输出高电平；若 $a < b$ ，则 alb 输出高电平。其仿真波形如图7：一组局部放大的测试数据和各位的含

义均已标注在波形图上。

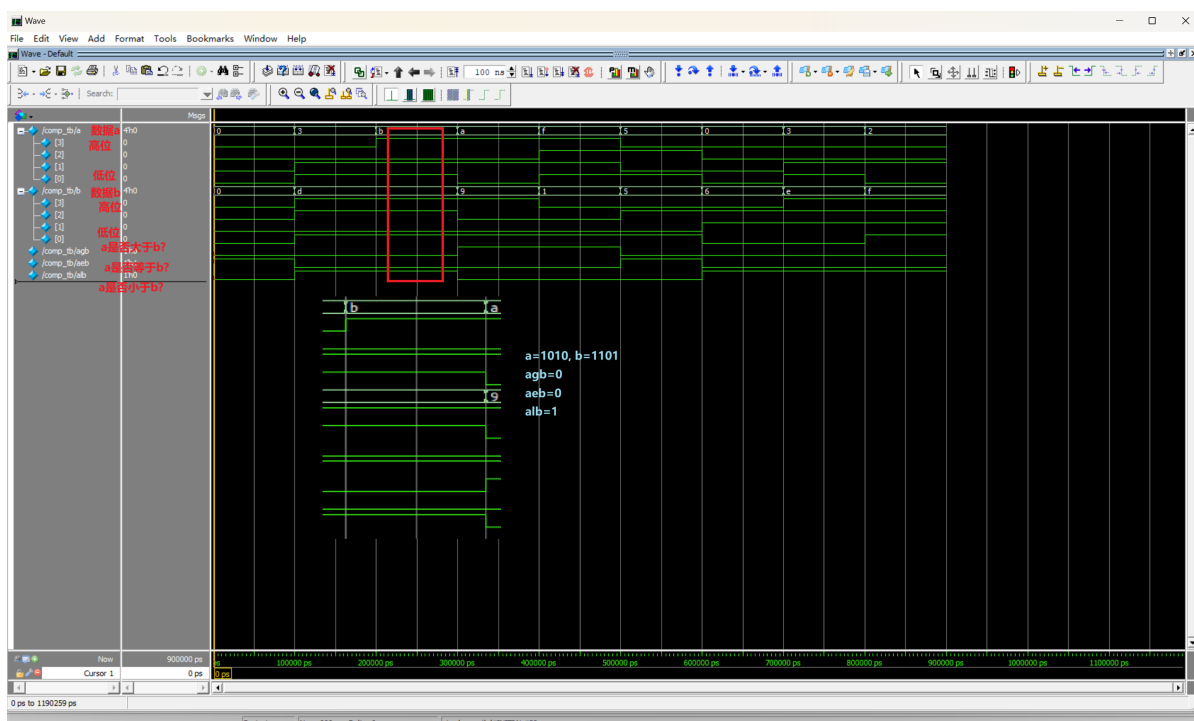


图 7: 比较器仿真结果

5.2 选择器模块

选择器的功能为：根据地址 $addr$ 选择输出两个数 In_0 和 In_1 ，如果 $addr=0$ 则选择 In_0 ， $addr=1$ 则选择 In_1 。仿真结果如图9。

5.3 全加器模块

全加器的功能为：输入两个一位二进制数 a 、 b ，以及上一位的进位 ci ，输出三个数相加的结果，其中 s 用于存储相加结果的个位， co 用于存储相加结果的十位（也就是下一位的进位）。全加器的仿真结果如图9。

5.4 两数之差的绝对值电路

任务一的顶层设计要求为：计算输入的两个四位二进制数之差的绝对值。其仿真结果如图10，运行仿真时的输出如图11。

5.5 模式比较器电路

任务二的顶层设计要求为：设计模式比较器，电路的输入为两个 8 位无符号二进制数 a 、 b 和一个模式控制信号 m ；电路的输出为 8 位无符号二进制数 y 。当 $m=0$ 时， $y=MAX(a,b)$ ；而当 $m=1$ 时，则 $y=MIN(a,b)$ 。其仿真结果如图12，运行仿真时的输出如图13。

综上所述，可见各模块单独工作时功能基本正确，两个顶层设计也较为合理，整体电路设计没有明显漏洞。

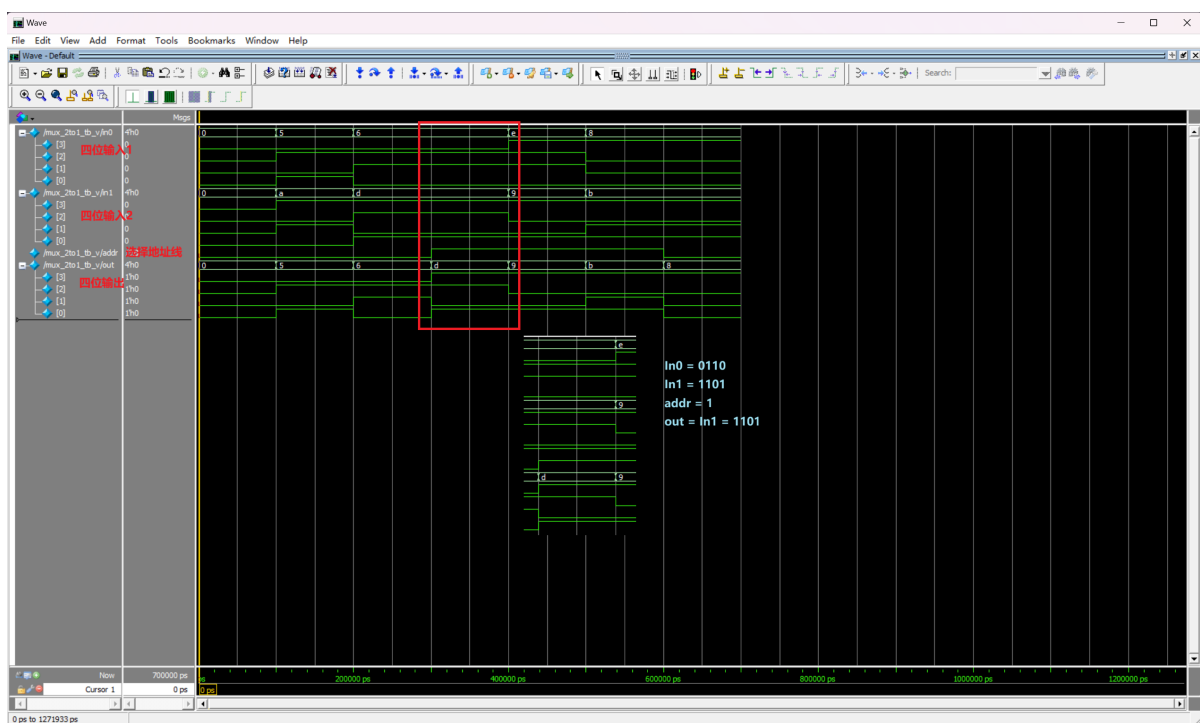


图 8: 选择器仿真结果

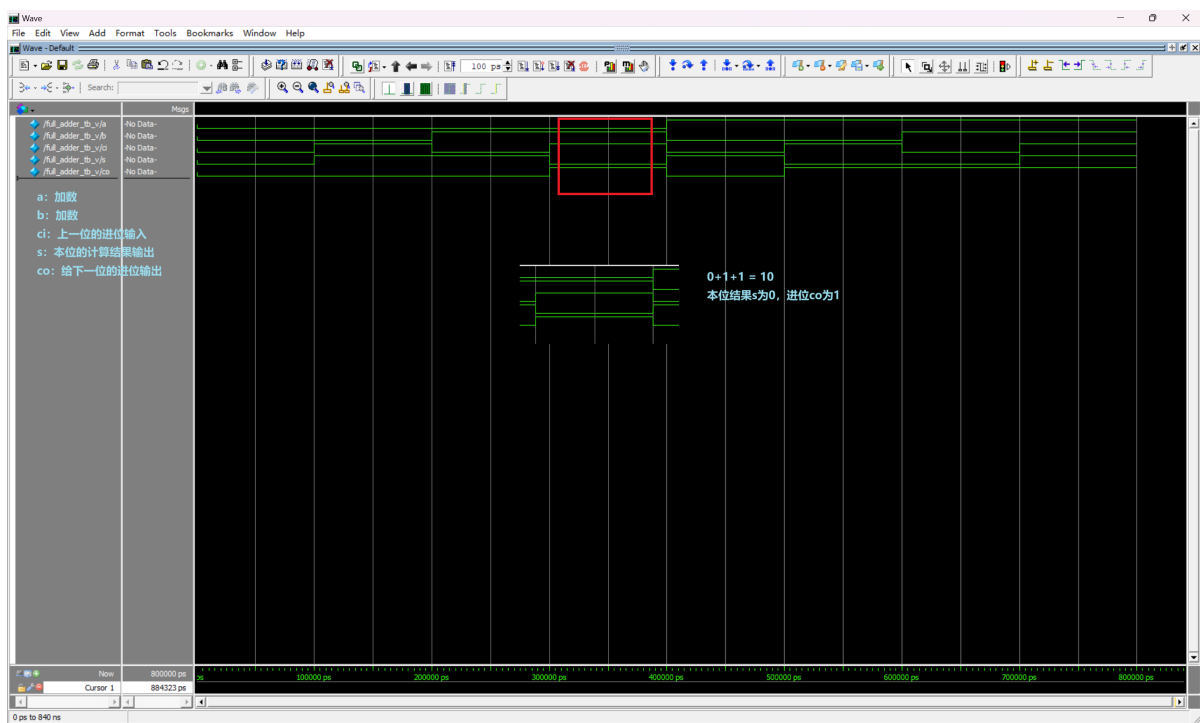


图 9: 全加器仿真结果

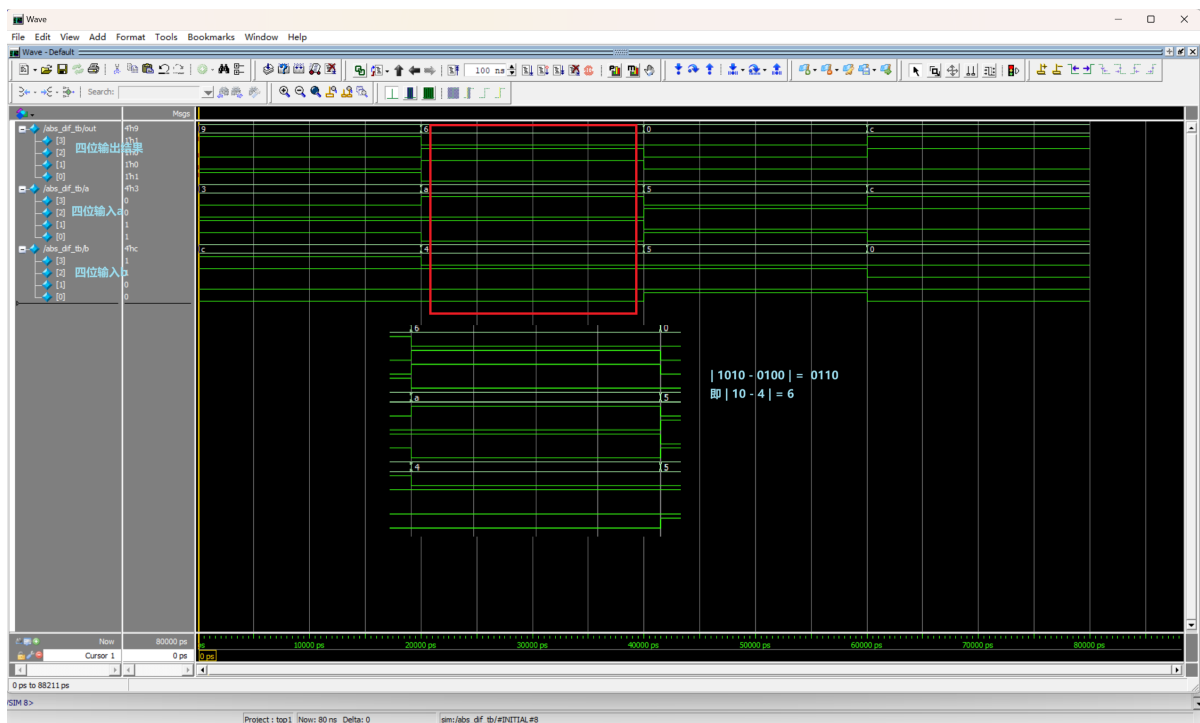


图 10: 两数之差的绝对值电路仿真结果

```
sim:/abs_diff_tb/out \
sim:/abs_diff_tb/a \
sim:/abs_diff_tb/b
VSIM3> run -all
# | 3 - 12 |= 9
# | 10 - 4 |= 6
# | 5 - 5 |= 0
# | 12 - 0 |= 12
# ** Note: $stop : D:/DigitalDesignLabs/lab5_combination/src/abs_diff_tb.v(18)
# Time: 80 ns Iteration: 0 Instance: /abs_diff_tb
# Break in Module abs_diff_tb at D:/DigitalDesignLabs/lab5_combination/src/abs_diff_tb.v line 18
```

图 11: 两数之差的绝对值电路输出结果

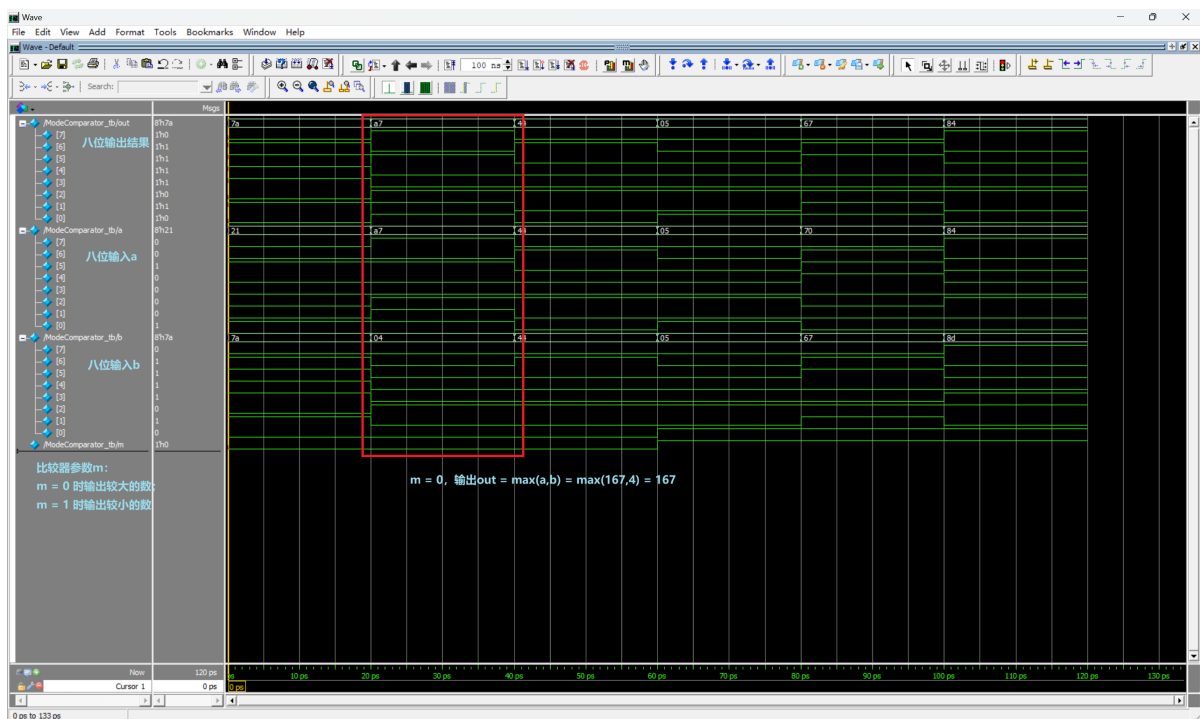


图 12: 模式比较器仿真结果

```
add wave \
sim:/ModeComparator_tb/out \
sim:/ModeComparator_tb/a \
sim:/ModeComparator_tb/b \
sim:/ModeComparator_tb/m
VSIM 22> run -all
# MAX( 33 , 122 )= 122
# MAX( 167 , 4 )= 167
# MAX( 68 , 68 )= 68
# MIN( 5 , 5 )= 5
# MIN( 112 , 103 )= 103
# MIN( 132 , 141 )= 132
# ** Note: $stop : D:/DigitalDesignLabs/lab5_combination/src/ModeComparator_tb.v(27)
# Time: 120 ps Iteration: 0 Instance: /ModeComparator_tb
# Break in Module ModeComparator_tb at D:/DigitalDesignLabs/lab5_combination/src/ModeComparator_tb.v line 27
```

图 13: 模式比较器仿真结果

六、实验结果分析、总结和心得

分析上述仿真结果，发现各模块单独工作时功能基本正确，整体电路设计无明显漏洞；分析 FPGA 实物电路的工作状态，发现对于不同测试数据均能正确输出计算结果，因此可以得出结论：本次实验完成了“设计计算两数之差的绝对值电路”和“设计模式比较器”的任务，且设计流程符合“层次设计”的思想，文件管理基本符合规范，整体实验较为圆满。

本次实验深刻体验了“自顶而下”、“层次设计”的设计思路，即先进行底层设计、封装，再调用底层模块进行上一层的设计，如此逐步累积形成一个大的项目的数字电路设计。这样的思想在之后工作时不管是设计电路还是调试、测试电路都十分有用，对于数字电路、模拟电路都十分有效。

七、思考题

7.1 求两数之差的绝对值电路，若输入为有符号数（补码），该怎样设计？请画出原理框图并作简要说明。

用补码表示负数的方法为求其补码 +1；故可以将减法用加法器实现，只需要将其中一个输入变成原来的相反数；再将输出求绝对值，可以根据其最高位表示符号的数作为数据选择器的地址线，数据选择器的一个输入为该数本身，另一个输入为该数的相反数，经数据选择器即可保证选取正数。

原理框图如图14：

7.2 能否用加法器实现比较器，若能，请画出原理框图。

可以用加法器实现比较器，方法如下：分析逻辑表达式，有

$$a < b = \bar{a}b = a(a \oplus b); a = b = a \oplus b; a > b = b(a \oplus b)$$

故可以画出原理框图，如图15

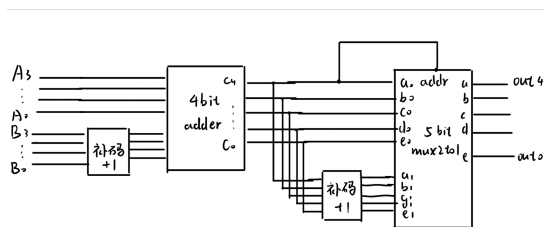


图 14：有符号数的差的绝对值电路框图

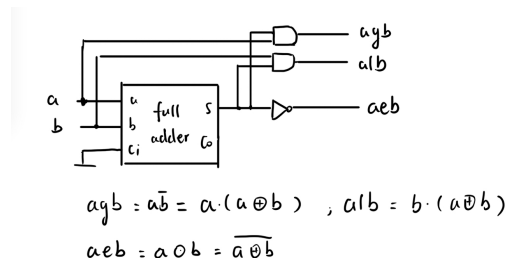


图 15：加法器实现比较器的电路框图

7.3 调用模块时怎样进行参数传递？若模块有多个参数，模块实例的格式如何？

格式如下代码所示，外层为模块定义的参数名，括号内为实际电路中的参数名。

```
1 comp #(8) comp1(.a (a),.b (b),.agb(agb),.aeb(aeb),.alb(alb));
2 mux_2to1 #(8) mux_min (.in0 (a),.in1 (b),.addr(agb),.out (min));
3 mux_2to1 #(8) mux_max (.in0 (a),.in1 (b),.addr(alb),.out (max));
4 mux_2to1 #(8) mux_out (.in0 (max),.in1 (min),.addr(m),.out (y));
```